

Reading Comprehension com RoBERTa em MultiRC

1st Rafael Ventura, 2nd André Fonseca

Departamento de Informática, Faculdade de Ciências (FCUL), Universidade de Lisboa (ULisboa), Lisboa, Portugal

fc66132@alunos.ciencias.ulisboa.pt, fc58225@alunos.ciencias.ulisboa.pt

Abstract—Este trabalho apresenta o desenvolvimento de um sistema *Reading Comprehension* sobre o dataset MultiRC (*Multi-Sentence Reading Comprehension*), pertencente ao benchmark SuperGLUE. A tarefa consiste numa classificação binária: dado um parágrafo, uma pergunta e uma resposta candidata, o modelo determina se a resposta é correta (1) ou incorreta (0). A abordagem adotada baseia-se no *fine-tuning* do modelo pré-treinado RoBERTa-base, utilizando a arquitetura `RobertaForSequenceClassification` com duas classes de saída. O treino foi realizado durante uma época com *batches* de dimensão 8 como pedido no enunciado do projeto, recorrendo ao otimizador AdamW. Os resultados obtidos foram 71,98% de *accuracy* no conjunto de treino e 69,49% no conjunto de validação, evidenciando uma generalização aceitável do modelo para dados não vistos com apenas 3406 *stpes*.

I. ESTRUTURA E FUNCIONAMENTO DO SOFTWARE

O trabalho desenvolvido implementa o *fine-tuning* do modelo pré-treinado `roberta-base` [1] para a tarefa de *Reading Comprehension* do dataset MultiRC (SuperGLUE) [2], tratada como um problema de classificação binária.

A estrutura deste projeto está organizada da seguinte forma: repositório público [3] para controlo de versões e persistência; desenvolvimento de um *jupyter notebook* para execução de todo o *pipeline* implementando - *fine-tuning*, treino e exportação do modelo.

A. Dataset

O dataset `aps/super_glue` [2] (configuração `multirc`) é carregado através da biblioteca `datasets` da Hugging Face, contendo três conjuntos: treino (27243 exemplos), validação (4848 exemplos) e teste (9693 exemplos). O conjunto de teste não é utilizado para avaliação uma vez que os seus labels não são públicos. Os exemplos existentes neste conjunto servem apenas para gerar previsões a submeter ao servidor oficial do benchmark SuperGLUE.

B. Tokenização

Para cada exemplo, o par de entrada é construído concatenando:

- **Sequência A:** o parágrafo (*paragraph*);
- **Sequência B:** a concatenação da pergunta com a resposta candidata (*question + answer*).

O `RobertaTokenizer` utiliza uma versão a nível de bytes do algoritmo *Byte-Pair Encoding* (BPE), com um vocabulário de 50,000 tokens. Esta abordagem de tokenização

por subpalavras decompõe palavras raras em unidades mais frequentes, evitando tokens desconhecidos e tornando o modelo robusto a variações morfológicas [4]. As sequências de entrada são limitadas a 512 tokens contíguos, com *truncation* ativada para respeitar esse limite [1]. O início de cada documento é marcado com `<s>` e o fim com `</s>`, pelo que o par de sequências fica com o formato:

`<s> parágrafo </s></s> pergunta resposta </s>`

O procedimento de *masking* descrito no pré-treino do RoBERTa [1], onde 15% dos tokens são mascarados dinamicamente a cada época, sendo 80% substituídos por `<mask>`, 10% por um token aleatório e 10% mantidos inalterados. Durante o *fine-tuning* realizado neste projeto, não é aplicado qualquer mascaramento.

O *padding* não é fixo na tokenização. Em vez disso, é utilizado um `DataCollatorWithPadding` que aplica *padding* dinâmico no momento de criação de cada *batch*, ao comprimento da sequência mais longa desse *batch*, reduzindo o consumo de memória. As colunas de texto originais são removidas após a tokenização, ficando cada exemplo representado por `input_ids`, `attention_mask` e `labels`.

C. Modelo

O modelo utilizado é o RoBERTa-base (Robustly Optimized BERT Pretraining Approach), um modelo da família Transformer pré-treinado pela Meta AI sobre grandes volumes de texto em inglês. O RoBERTa-base partilha a mesma arquitetura do BERT-base (12 camadas ocultas (L=12), 12 cabeças de atenção (A=12) e um *Hidden Size* de 748) [5]. O modelo BERT-base contém um total de 110 milhões de parâmetros [5], já o RoBERTa-base conta com 125 milhões, ou seja, um acréscimo de 15 milhões de parâmetros devido à adoção de um vocabulário BPE (Byte-Pair Encoding) ao nível do byte substancialmente maior. Enquanto a implementação original do BERT utiliza um vocabulário BPE ao nível do carácter com um tamanho de 30 mil unidades (aprendido após um pré-processamento com regras heurísticas de tokenização), o RoBERTa utiliza um vocabulário alargado de 50 mil unidades de subpalavras ao nível do byte, não requerendo qualquer pré-processamento ou tokenização adicional do input [6].

A biblioteca HuggingFace Transformers disponibiliza duas formas de instanciar o RoBERTa para uma tarefa de classificação. A primeira, através de `RobertaModel`, carrega

apenas o encoder base e devolve os hidden states de todos os tokens, exigindo a necessidade de construção manual de uma cabeça de classificação, incluindo as camadas lineares, a função de ativação, o dropout e o cálculo explícito da função de custo. A segunda, através de RobertaForSequenceClassification, estende o encoder com uma cabeça de classificação integrada: o *hidden state* do token especial <s> (equivalente ao [CLS] do BERT) é passado por duas camadas lineares com ativação *tanh* e *dropout*, produzindo os logits para as classes de saída. A função de custo é a CrossEntropyLoss, calculada automaticamente quando os labels são fornecidos.

Para o nosso projeto adotamos a abordagem RobertaForSequenceClassification, por ser a forma recomendada pela HuggingFace para tarefas de classificação de seqüências, por se integrar diretamente com a API Trainer sem necessidade de código adicional para o ciclo de treino e porque foi uma abordagem utilizada anteriormente nesta cadeia. A cabeça de classificação foi configurada com `num_labels=2`, correspondendo às classes 0 (resposta incorreta) e 1 (resposta correta) do dataset MultiRC.

D. Treino

O treino é gerido pela classe `Trainer` da Hugging Face, que trata automaticamente do ciclo de treino, *backpropagation*, avaliação periódica e gravação do melhor modelo (com base na *accuracy* de validação).

E. Função test

Foi implementada a função `test(passage, question, answer)`, que:

- 1) Recebe um contexto, uma pergunta e uma resposta candidata;
- 2) Tokeniza o par (parágrafo, pergunta + resposta) da mesma forma que no treino;
- 3) Faz *forward pass* no modelo afinado, no modo de avaliação (`eval()`), sem cálculo de gradientes);
- 4) Retorna a classe prevista (0 - resposta incorreta, ou 1 - resposta correta), obtida através do `argmax` dos logits.

II. TRAINING BUDGET

TABLE I: Comparative Framework: Cooperative vs. Competitive

Hiperparâmetro	Valor
Modelo	roberta-base
Número de classes	2
Epochs	1
Batch size	8
Learning rate (LR)	2×10^{-5}
Otimizador	AdamW (padrão)
Função de custo	Cross-Entropy (padrão)

Os valores de *batch size* (8) e número de *epochs* (1) foram dados pelo enunciado. O *learning rate* e o *warmup ratio* foram ajustados experimentalmente, uma vez que com o valor *default* do `Trainer` (5×10^{-5}) o modelo não convergia (*accuracy* próxima de 50%, ou seja, era apenas uma escolha aleatória).

III. RESULTADOS

TABLE II: Comparative Framework: Cooperative vs. Competitive

Conjunto	Accuracy (%)
Treino	0.719
Validação	0.695
Teste	Not recorded

A. Comparação de learning rates

De forma a justificar a escolha do *learning rate* (2×10^{-5}), foram realizadas experiências adicionais com outros valores, mantendo todos os restantes hiperparâmetros fixos (1 *epoch*, *batch size* 8). Os resultados são apresentados na Tabela III.

TABLE III: Comparação de diferentes learning rates

Learning rate	Train Loss	Train Acc.	Val Acc.
1×10^{-2}	1.070	0.559	0.572
5×10^{-5} (padrão)	0.686	0.559	0.572
2×10^{-5}	0.664	0.719	0.695

Tanto com $lr = 10^{-2}$ como com o valor *default* (5×10^{-5}) [7], o modelo não consegue aprender, convergindo para a previsão sistemática da classe majoritária (*accuracy* ≈ 0.572 , correspondente à proporção da classe predominante no *validation set*). No caso de $lr = 10^{-2}$, a *loss* de treino chega a atingir valores muito elevados (≈ 1.39) nos primeiros *steps*, evidenciando instabilidade no início do treino, antes de estabilizar num mínimo trivial. Apenas com $lr = 2 \times 10^{-5}$ o modelo efetivamente aprendeu a tarefa, o que justifica a escolha deste valor para a configuração final.

Nota sobre o conjunto de teste: O conjunto de teste do MultiRC, tal como distribuído no SuperGLUE, não inclui *labels* públicas - estas são reservadas para submissão na plataforma oficial do benchmark. Como tal, não é possível calcular a *accuracy* no conjunto de teste. A função `test` foi, no entanto, validada com exemplos do conjunto de validação, para os quais as *labels* estão disponíveis, confirmando o correto funcionamento do modelo afinado.

A pequena diferença entre a *accuracy* de treino (71.98%) e de validação (69.49%) indica que o modelo não sofreu *overfitting* significativo, isto é, não memorizou os dados de treino e foi capaz de generalizar muito bem (acc. $\approx 50\%$) para apenas uma *epoch* de treino. Analisando os resultados não podemos afirmar que o modelo iria performar igualmente bem para o *test set*, contudo os resultados do *validation set* sugerem possíveis resultados otimistas numa fase de teste.

IV. REQUISITOS IMPLEMENTADOS

Todas as funcionalidades pedidas no enunciado foram implementadas:

- *Fine-tuning* do roberta-base no MultiRC;
- Função `test(passage, question, answer)`;
- Gravação local do modelo afinado;
- Impressão das métricas de *accuracy* de treino e validação.

V. ERROS CONHECIDOS

Não foram identificados erros relevantes na versão final do código. Durante o desenvolvimento foram resolvidos os seguintes problemas técnicos:

- Incompatibilidade entre versões do `torchvision` e `datasets` no Google Colab (resolvida com a reinstalação do `torchvision`);
- *Learning rate default* demasiado elevado, que impedia a convergência do modelo (resolvido com $lr = 2 \times 10^{-5}$ e *warmup ratio*= 0.1).

VI. CONTRIBUIÇÃO DOS MEMBROS DO GRUPO

TABLE IV: Comparative Framework: Cooperative vs. Competitive

Membro	Contributo (%)	Tempo despendido
André Fonseca	50	$\approx 10h$
Rafael Ventura	50	$\approx 10h$

Ambos os elementos colaboraram em conjunto em todas as fases do projeto: implementação do pipeline de tokenização e treino, ajuste de hiperparâmetros, implementação da função `test`, execução no Google Colab e redação do presente relatório.

REFERENCES

- [1] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “Roberta: A robustly optimized BERT pretraining approach,” *CoRR*, vol. abs/1907.11692, 2019.
- [2] A. Wang, Y. Pruksachatkun, N. Nangia, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, “Superglue: A stickier benchmark for general-purpose language understanding systems,” 2020.
- [3] R. Ventura and A. Fonseca, “rafaelgm/roberta-multirc-reading-comprehension: v1.0.0,” <https://doi.org/10.5281/zenodo.20681263>, 2026. Zenodo DOI: 10.5281/zenodo.20681263.
- [4] H. Face, “Byte-pair encoding tokenization.”
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” 2019.
- [6] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “Roberta: A robustly optimized bert pretraining approach,” 2019.
- [7] H. Face, “Trainer.”